

Nama : Neva Maritza Andini

Kelas : A

NIM : 242410101021

Praktikum 1 AJAX + Cookie + Session

1. Buat section yang menampilkan data cuaca kota Surabaya menggunakan API publik <https://wttr.in/Surabaya?format=json> (gratis, tidak perlu API key). Tampilkan: nama kota, suhu saat ini (dalam °C), dan deskripsi cuaca. Gunakan async/await dan tampilkan loading indicator saat data sedang diambil.

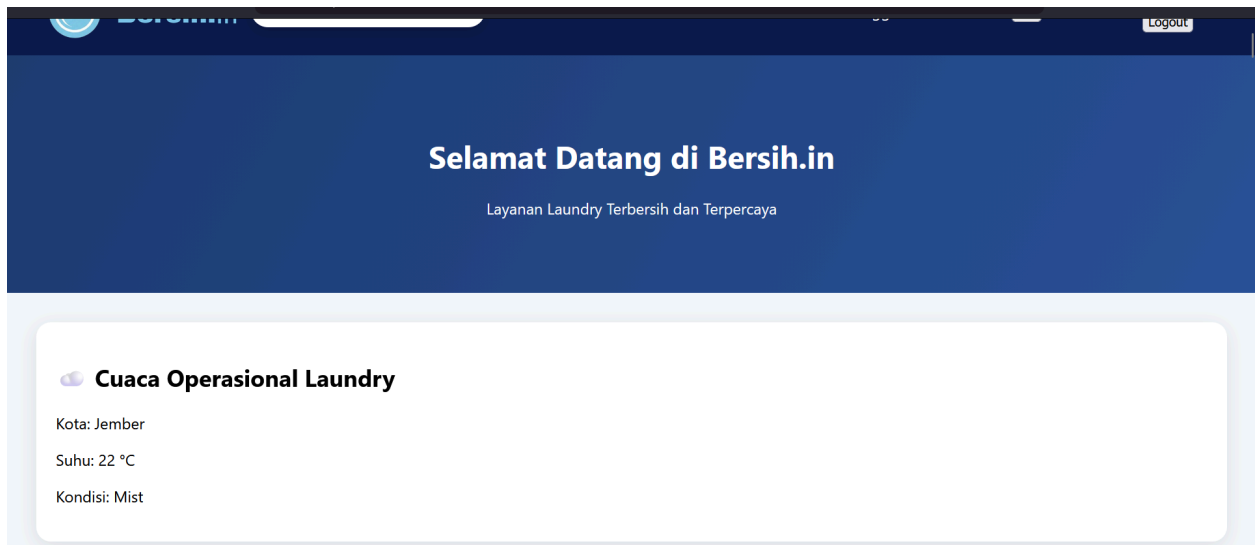
Bisa menggunakan api publik yang sesuai proyek masing masing

Jawaban:

```
Controller.php M X | stat-card.blade.php | app.blade.php M | navbar.blade.php M | foot
resources > views > dashboard.blade.php > script
1 @extends('layouts.app')
2 @section('title', 'Dashboard')
3 @section('content')
4
5 <section class="hero">
6     <h1>Selamat Datang di Bersih.in</h1>
7     <p>Layanan Laundry Terbersih dan Terpercaya</p>
8 </section>
9
10 <section class="weather-card">
11     <h2>🌤️ Cuaca Operasional Laundry</h2>
12     <div id="loading">
13         Memuat data cuaca...
14     </div>
15     <div id="weather" style="display:none">
16         <p>
17             Kota:
18             <span id="kota"></span>
19         </p>
20         <p>
21             Suhu:
22             <span id="suhu"></span> °C
23         </p>
24         <p>
25             Kondisi:
26             <span id="deskripsi"></span>
27         </p>
28     </div>
29 </section>
```

```
Controller.php M stat-card.blade.php app.blade.php M navbar.blade.php M footer.blade.php # style.css M JS tailwind.config.js M JS script.js
resources > views > dashboard.blade.php > script > ambilCuaca
111
112 <script>
113 async function ambilCuaca() {
114   try {
115     const response =
116       await fetch(
117         | 'https://wttr.in/Jember?format=j1'
118       );
119     if(!response.ok){
120       throw new Error();
121     }
122     const data =
123       | await response.json();
124     document
125       .getElementById('kota')
126       .innerText =
127         'Jember';
128     document
129       .getElementById('suhu')
130       .innerText =
131         data.current_condition[0].temp_C;
132     document
133       .getElementById('deskripsi')
134       .innerText =
135         data.current_condition[0]
136         | .weatherDesc[0]
137         | .value;
138     document
139       .getElementById('loading')
140       .style.display =
141         'none';
142     document
143       .getElementById('weather')
144       .style.display =
145         'block';
146   }
147   catch(error){
148     document
149       .getElementById('loading')
150       .innerText =
```

Tampilan:



Penjelasan: Pada dashboard aplikasi BersihIn untuk menampilkan kondisi cuaca kota Surabaya menggunakan API publik wttr.in tanpa memerlukan API key. Informasi yang ditampilkan meliputi nama kota, suhu saat ini (°C), dan deskripsi cuaca. Implementasi dilakukan menggunakan JavaScript Fetch API dengan pendekatan async/await agar proses pengambilan data berjalan secara

asynchronous tanpa melakukan reload halaman. Fungsi async function `ambilCuaca()` digunakan untuk menjalankan proses pengambilan data, kemudian `await fetch()` digunakan untuk mengirim request ke API dan `await response.json()` digunakan untuk mengubah response menjadi format JSON. Data yang diperoleh kemudian ditampilkan ke halaman menggunakan `document.getElementById().innerText`. Selama proses pengambilan data berlangsung ditampilkan loading indicator berupa teks “Memuat data cuaca...” yang akan disembunyikan setelah data berhasil dimuat. Selain itu, digunakan try-catch untuk menangani kondisi ketika terjadi kegagalan saat mengambil data sehingga sistem dapat menampilkan pesan error kepada pengguna.

2. Buat live search yang memiliki form sederhana untuk mencari mahasiswa. Form dikirim via AJAX POST/ GET ke Laravel halaman tidak boleh reload sama sekali. Setelah berhasil, catatan baru langsung muncul di daftar bawah tanpa refresh. Gunakan CSRF token yang benar.

Jawaban:

```
web.php M X DashboardController.php M PreferensiController.p
routes > web.php
37 Route::middleware('auth')->group(function(){
38     Route::resource(
39         'transaksi',
40         TransaksiController::class
41     );
42     Route::get(
43         '/search-transaksi',
44         [TransaksiController::class, 'search']
45     );
46     Route::get(
47         '/profile',
48         [ProfileController::class, 'edit']
49     )->name('profile.edit');
50     Route::patch(
51         '/profile',
52         [ProfileController::class, 'update']
53     )->name('profile.update');
54
55     Route::delete(
56         '/profile',
57         [ProfileController::class, 'destroy']
58     )->name('profile.destroy');
59
60 });
```

```

8 class TransaksiController extends Controller
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64 public function search(Request $request)
65 {
66     $cari =
67     $request->q;
68     $data =
69
70     Transaksi::where(
71         'user_id',
72         auth()->id()
73     )
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95     ->where(
96         'nama_pelanggan',
97         'like',
98         "%$cari%"
99     )
100
101
102
103     ->get();
104     return response()
105     ->json(
106         $data
107     );
108 }

```

oller.php M stat-card.blade.php app.blade.php M navbar

resources > views > dashboard.blade.php > script > ambilCuaca

```

30
31 <section class="search-section">
32     <h2>
33         Cari Transaksi
34     </h2>
35     <input
36         type="text"
37         id="search"
38         placeholder="Cari pelanggan..."
39     >
40     <div id="hasil"></div>
41 </section>
42

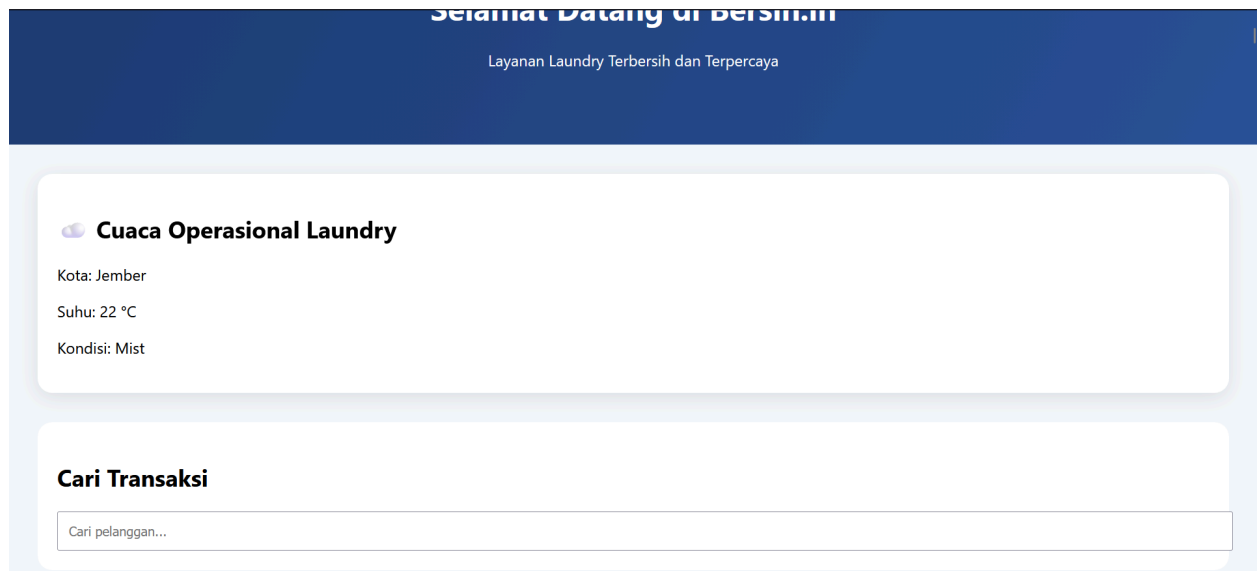
```

oller.php M X stat-card.blade.php app.blade.php M navbar.blade.php M footer.blade.php # style.css M JS

resources > views > dashboard.blade.php > script > ambulCuaca

```
112 <script>
207 const search =
208     document
209     | .getElementById(
210     |     'search'
211     | );
212
213 search
214     .addEventListener(
215     | 'input',
216     | async function(){
217     |     try{
218     |         let keyword =
219     |         | this.value;
220     |         if(keyword===''){
221     |             document
222     |             | .getElementById(
223     |             |     'hasil'
224     |             | )
225     |             | .innerHTML='';
226     |             return;
227     |         }
228     |
229     |         const response =
230     |         | await fetch(
231     |         |     `/search-transaksi?q=${keyword}`
232     |         | );
233     |         if(!response.ok){
234     |             throw new Error();
235     |         }
236     |
237     |         const data =
238     |         | await response.json();
239     |         if(data.length===0){
240     |             document
241     |             | .getElementById(
242     |             |     'hasil'
243     |             | )
244     |             | .innerHTML=
245     |             | 'Tidak ditemukan';
```

Tampilan:



Selamat Datang di BersihIn

Layanan Laundry Terbersih dan Terpercaya

Cuaca Operasional Laundry

Kota: Jember

Suhu: 22 °C

Kondisi: Mist

Cari Transaksi

Cari pelanggan...

Penjelasan: Pada fitur ini dibuat live search transaksi pelanggan pada dashboard aplikasi BersihIn dengan memanfaatkan AJAX menggunakan Fetch API sehingga proses pencarian dapat dilakukan tanpa me-refresh halaman. Fitur ini merupakan penyesuaian dari studi kasus pencarian mahasiswa menjadi pencarian transaksi pelanggan sesuai kebutuhan proyek.

Implementasi dimulai dengan menambahkan route pencarian pada Laravel yang bertugas menerima request dari JavaScript dan mengarahkan proses ke method search() pada TransaksiController. Route ini dilindungi menggunakan middleware auth sehingga hanya pengguna yang sudah login yang dapat mengakses fitur pencarian.

Pada bagian controller dibuat method search(Request \$request) untuk menerima input pencarian menggunakan \$request->q. Data transaksi kemudian dicari menggunakan query where() berdasarkan user yang sedang login dan nama pelanggan yang sesuai dengan kata kunci pencarian. Hasil pencarian dikembalikan dalam format JSON menggunakan response()->json() agar dapat diproses oleh JavaScript.

Pada sisi frontend ditambahkan form pencarian sederhana menggunakan elemen input dan dihubungkan dengan Fetch API. Ketika pengguna mengetik pada kolom pencarian, event input akan mengirim request AJAX GET ke Laravel secara otomatis tanpa reload halaman. Data hasil pencarian diterima dalam format JSON, kemudian ditampilkan langsung ke bagian hasil menggunakan manipulasi DOM (innerHTML). Implementasi juga menggunakan CSRF Token untuk menjaga keamanan komunikasi antara frontend dan backend.

3. Buat navbar untuk mengeset dark mode.
 - a. Tambahkan dark mode ke konfigurasi Tailwind (darkMode: 'class' di [tailwind.config.js](#))
 - b. Buat helper function JavaScript: setCookie, getCookie, deleteCookie

- c. Implementasikan toggle dark mode: klik tombol → toggle class 'dark' di HTML → simpan ke cookie
- d. Terapkan cookie di awal load (dalam tag script di <head>) untuk mencegah flash of unstyled content
- e. Buat halaman pengaturan /preferensi dengan form: pilihan tema (light/dark/system), pilihan ukuran font
- f. Simpan form ke cookie via Fetch POST ke endpoint Laravel
- g. Laravel Controller: baca cookie dari \$request->cookie(), kembalikan JSON konfirmasi

Jawaban:

```

JS tailwind.config.js > default
1 import defaultTheme from 'tailwindcss/defaultTheme';
2 import forms from '@tailwindcss/forms';
3
4 /** @type {import('tailwindcss').Config} */
5 export default {
6   darkMode: 'class',
7   content: [
8     './vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php',
9     './storage/framework/views/*.php',
10    './resources/views/**/*.blade.php',
11  ],
12
13   theme: {
14     extend: {
15       fontFamily: {
16         sans: ['Figtree', ...defaultTheme.fontFamily.sans],
17       },
18     },
19   },
20
21   plugins: [forms],
22 };

```

a.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  DEVDB
PS C:\laragon\www\BersihIn> npm run dev
VITE v8.0.12 ready in 5155 ms

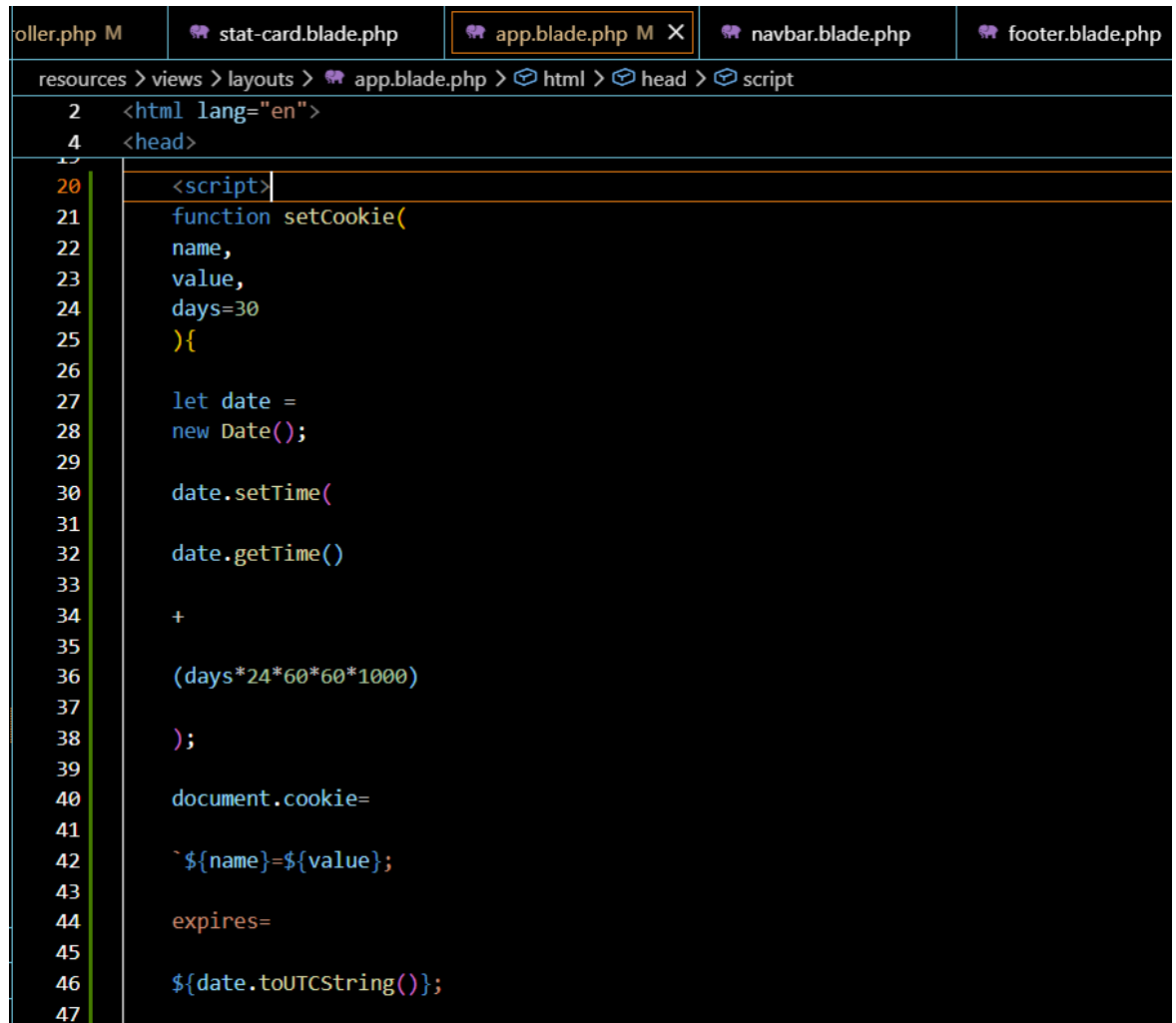
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

LARAVEL v13.6.0 plugin v3.1.0

→ APP_URL: http://localhost

```


Penjelasan: Tahap ini dilakukan konfigurasi Tailwind CSS dengan menambahkan properti `darkMode: 'class'` pada file `tailwind.config.js`. Konfigurasi ini memungkinkan aplikasi mengaktifkan tema gelap melalui penambahan class `dark` pada elemen HTML tanpa perlu mengganti stylesheet secara manual. Dengan pendekatan ini, perubahan tema dapat dikontrol menggunakan JavaScript dan diterapkan secara dinamis pada seluruh halaman aplikasi.



```
oller.php M | stat-card.blade.php | app.blade.php M X | navbar.blade.php | footer.blade.php
resources > views > layouts > app.blade.php > html > head > script
2 <html lang="en">
4 <head>
19
20 <script>
21 function setCookie(
22   name,
23   value,
24   days=30
25 ){
26
27   let date =
28   new Date();
29
30   date.setTime(
31
32   date.getTime()
33
34   +
35
36   (days*24*60*60*1000)
37
38   );
39
40   document.cookie=
41
42   `${name}=${value};
43
44   expires=
45
46   ${date.toUTCString()};
47
```

b.

```
oller.php M  stat-card.blade.php  app.blade.php M X  guest.blade.php

resources > views > layouts > app.blade.php > html > head > script

2  <html lang="en">
4  <head>
19  <script>

43      function getCookie(
44          name
45      ){
46
47          let data=
48          document.cookie
49          .split(';');
50
51          for(
52          let i=0;
53          i<data.length;
54          i++
55          ){
56
57              let c=
58              data[i]
59              .trim();
```

```
oller.php M  stat-card.blade.php  app.blade.php M X  guest.blade.php

resources > views > layouts > app.blade.php > html > head > script

2  <html lang="en">
4  <head>
19  <script>

79      function deleteCookie(
80          name
81      ){
82
83          document.cookie=
84          `${name}=;
85          expires=Thu, 01 Jan 1970 00:00:00 UTC;
86          path=/`;
87
88      }
89
```

Penjelasan: Beberapa helper function JavaScript untuk mengelola cookie pada browser. Fungsi `setCookie()` digunakan untuk menyimpan preferensi pengguna, `getCookie()` digunakan untuk membaca nilai cookie yang telah tersimpan, sedangkan `deleteCookie()` digunakan untuk menghapus data cookie. Implementasi

helper ini digunakan sebagai dasar penyimpanan preferensi tampilan pada aplikasi BersihIn.

```
139
140 <script>
141
142 document
143 .getElementById(
144   'toggleDark'
145 )
146
147 ?.addEventListener(
148
149   'click',
150
151   ()=>{
152
153     document
154     .documentElement
155     .classList
156     .toggle(
157       'dark'
158     );
159
160     if(
161
162       document
163       .documentElement
164       .classList
165       .contains(
166         'dark'
167       )
```

c.

```
oller.php M  stat-card.blade.php  app.blade.php M  guest.blade.php  navbar.blade.php M X
resources > views > partials > navbar.blade.php > ...
1  <header class="navbar">
37 </nav>
38 <button
39   id="toggleDark">
40
41   🌙
42
43 </button>
44   <div class="user-login">
45     Halo, {{ auth()->user()->name }}
46   </div>
47 <div class="user-section">
```

Penjelasan: Tombol toggle dark mode pada navbar aplikasi BersihIn. Ketika tombol ditekan, sistem akan menambahkan atau menghapus class dark menggunakan `classList.toggle()` sehingga tampilan halaman berubah antara mode terang dan mode gelap. Setelah tema berhasil diubah, nilai preferensi disimpan ke cookie agar tetap digunakan saat pengguna membuka kembali aplikasi.

```
oller.php M  stat-card.blade.php  app.blade.php M X  guest.blade.php    
resources > views > layouts > app.blade.php > html > head > script  
2  <html lang="en">  
4  <head>  
19  <script>  
89  
90      let mode=  
91      getCookie(  
92      'theme'  
93      );  
94  
95      if(  
96      mode=== 'dark'  
97      ){  
98  
99      document  
100      .documentElement  
101      .classList  
102      .add(  
103      'dark'  
104      );  
105  
106      }  
107  
108  </script>  
109
```

d.

Penjelasan: Sistem membaca nilai cookie tema saat halaman pertama kali dimuat menggunakan `getCookie()`. Jika ditemukan nilai `dark`, maka class `dark` langsung diterapkan pada elemen HTML sebelum halaman selesai dirender. Implementasi ini bertujuan untuk mencegah terjadinya flash of unstyled content, yaitu kondisi ketika halaman sempat tampil dengan tema terang sebelum berubah menjadi gelap.

```
web.php M X | preferensi.blade.php U | TransaksiController.php M | stat-card.blade.php

routes > web.php
46
47 Route::view(
48     '/preferensi',
49     'preferensi'
50 );
51
52 require __DIR__.'/auth.php';
53
```

e.

```
web.php M | preferensi.blade.php U X | TransaksiController.php M | stat-card.blade.php

resources > views > preferensi.blade.php > section.preferensi
1 @extends('layouts.app')
2 @section('title','Preferensi')
3 @section('content')
4
5 <section class="preferensi">
6 <h2>
7 Pengaturan Tampilan
8 </h2>
9
10 <form
11 id="prefForm">
12 <label>Tema</label>
13
14 <select
15 name="tema">
16 <option value="light">
17 | Light
18 </option>
19
20 <option value="dark">
21 | Dark
22 </option>
23
24 <option value="system">
25 | System
26 </option>
27 </select>
28 <br><br>
29 <label>
30 | Ukuran Font
```

Penjelasan: Halaman pengaturan preferensi dapat digunakan pengguna untuk mengatur tampilan aplikasi. Halaman ini menyediakan form dengan pilihan tema (light, dark, dan system) serta ukuran font (kecil, sedang, dan besar). Form tersebut menjadi media bagi pengguna untuk menyimpan preferensi tampilan sesuai kebutuhan.

```
web.php M X | preferensi.blade.php U | TransaksiController.php M | stat-card.blade.php
routes > web.php
1 <?php
2
3 use App\Http\Controllers\ProfileController;
4 use Illuminate\Support\Facades\Route;
5 use App\Http\Controllers\DashboardController;
6 use App\Http\Controllers\TransaksiController;
7 use App\Http\Controllers\PreferensiController;
8
9 Route::get('/', function () {
10 |     return view('welcome');
11 });
12
13 Route::get('/dashboard', function () {
14 |     return view('dashboard');
15 });
16 ->middleware(['auth', 'verified'])
17 ->name('dashboard');
18
19 Route::middleware('auth')->group(function(){
20
21 Route::resource(
22 |     'transaksi',
23 |     TransaksiController::class
24 );
25
26 Route::get(
27 |     '/search-transaksi',
28 |     [TransaksiController::class, 'search']
29 );
30
31 Route::get(
32 |     '/profile',
33 |     [ProfileController::class, 'edit']
34 )->name('profile.edit');
35
36 Route::patch(
37 |     '/profile',
38 |     [ProfileController::class, 'update']
39 )->name('profile.update');
40
41 Route::delete(
42 |     '/profile',
43 |     [ProfileController::class, 'destroy']
44 )->name('profile.destroy');
45
46 });
47
48 Route::view(
49 |     '/preferensi',
50 |     'preferensi'
51 );
52
53 Route::post(
54 |     '/preferensi',
55 |     [
56 |         PreferensiController::class,
57 |         'store'
58 |     ]
59 );
```

f.


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVD8
PS C:\laragon\www\BersihIn>
* History restored
PS C:\laragon\www\BersihIn> php artisan make:controller PreferensiController
INFO Controller [C:\laragon\www\BersihIn\app\Http\Controllers\PreferensiController.php] created successfully.
PS C:\laragon\www\BersihIn>
```

```
web.php M DashboardController.php X PreferensiController.php U X preferensi.blade.php U
app > Http > Controllers > PreferensiController.php > PreferensiController > store
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6
7 class PreferensiController extends Controller
8 {
9     public function store(
10         Request $request
11     ){
12
13         return response()
14             ->json([
15
16             'status'=>
17             'berhasil'
18
19         ]);
20
21     }
22 }
23
```

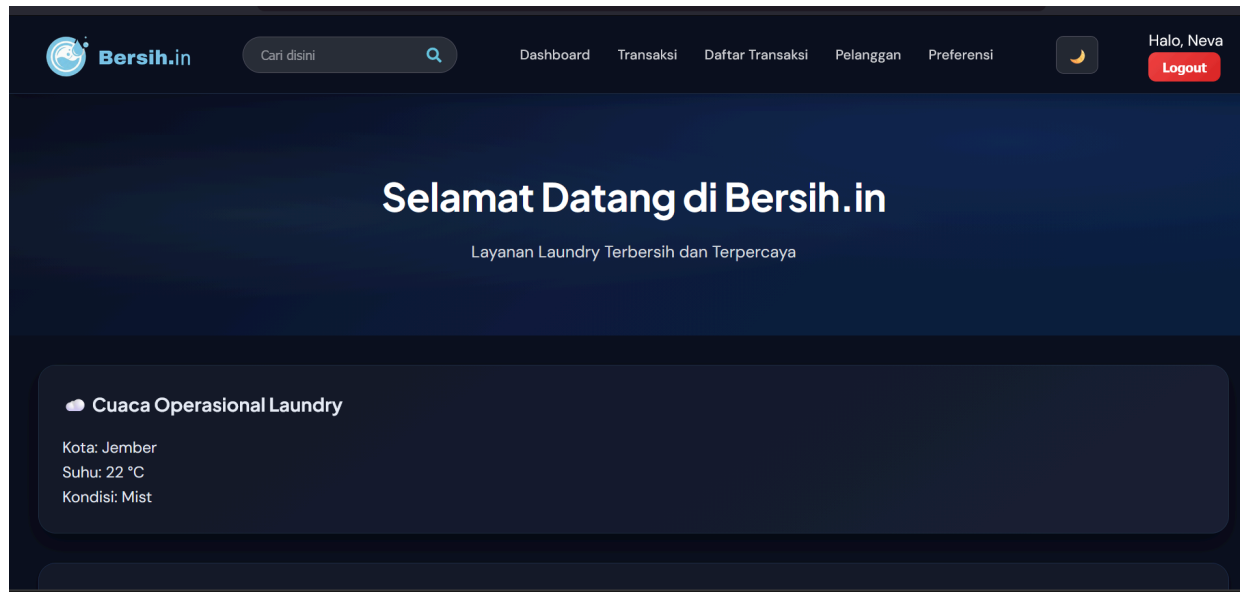
```
web.php M DashboardController.php PreferensiController.php U preferensi.blade.php U X TransaksiControlle
resources > views > preferensi.blade.php > ...
30
31 @push('scripts')
32 <script>
33     document.getElementById('prefForm').addEventListener('submit', async function(e) {
34         e.preventDefault();
35
36         try {
37             let data = new FormData(this);
38
39             const response = await fetch('/preferensi', {
40                 method: 'POST',
41                 headers: {
42                     'X-CSRF-TOKEN': '{{ csrf_token() }}'
43                 },
44                 body: data
45             });
46
47             if (!response.ok) {
48                 throw new Error();
49             }
50
51             setCookie('theme', data.get('tema'));
52             setCookie('font', data.get('font'));
53
54             alert('Preferensi tersimpan');
55         } catch {
56             alert('Gagal menyimpan');
57         }
58     });
59 </script>
60 @endpush
```

Penjelasan: Fetch API dengan metode POST untuk mengirim data dari form preferensi ke endpoint Laravel tanpa melakukan reload halaman. Data yang dikirim berupa pilihan tema dan ukuran font. Implementasi juga menggunakan CSRF Token untuk menjaga keamanan proses pengiriman data dari frontend ke backend. Setelah data diterima, preferensi disimpan ke cookie agar dapat digunakan kembali saat halaman dibuka.

```
web.php M DashboardController.php PreferensiController.php U X preferensi.blade.php U
app > Http > Controllers > PreferensiController.php > PreferensiController
1  <?php
2
3  namespace App\Http\Controllers;
4  use Illuminate\Http\Request;
5
6  class PreferensiController extends Controller
7  {
8
9      public function store(
10         Request $request
11     ) {
12
13         return response()
14             ->json([
15
16                 'status' =>
17                     'Preferensi berhasil disimpan',
18
19                 'tema' =>
20
21                     $request
22                         ->cookie(
23                             | 'theme'
24                         ),
25
26                 'font' =>
27
28                     $request
29                         ->cookie(
30                             | 'font'
31                         )
32             ]);
33
34 }
```

g.

Tampilan:



Penjelasan: Controller Laravel untuk membaca nilai cookie menggunakan `$request->cookie()` dan mengembalikan hasil dalam format JSON sebagai konfirmasi bahwa preferensi berhasil diproses. Data yang dibaca meliputi preferensi tema dan ukuran font yang telah dipilih pengguna. Implementasi ini menunjukkan integrasi antara frontend dan backend dalam pengelolaan preferensi tampilan aplikasi.

4. Buat section yang menghitung berapa kali pengguna mengunjungi halaman tersebut menggunakan session Laravel. Tampilkan: jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Sediakan tombol "Reset Hitungan" yang menghapus data dari session dan mengulang dari awal.

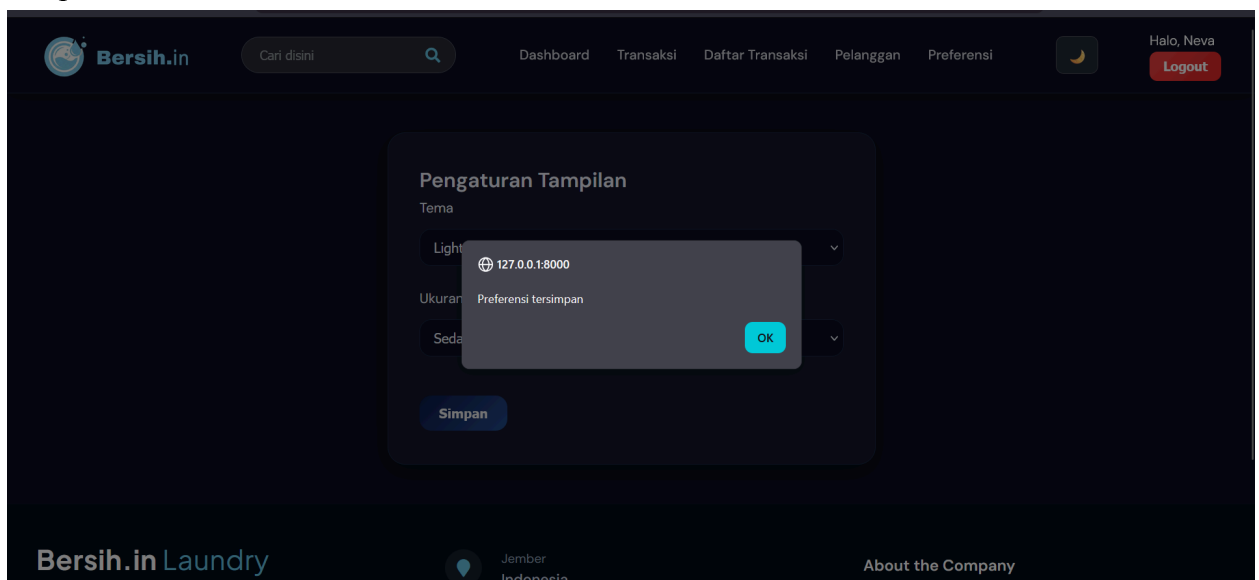
Bisa menggunakan case cart, sesuaikan dengan proyek

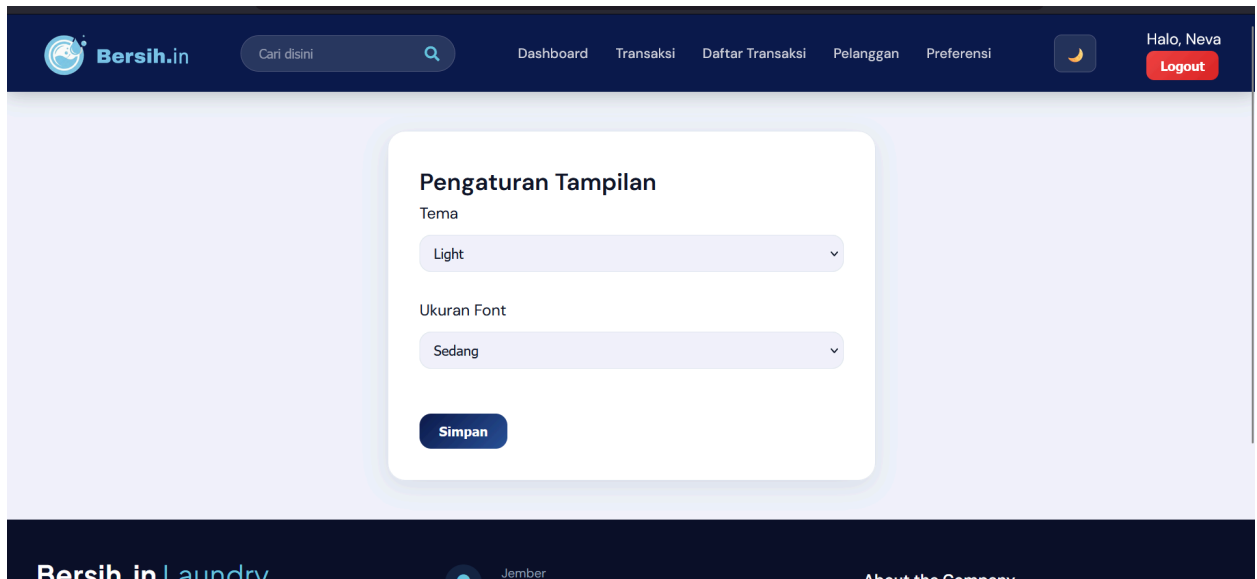
Jawaban:

```
web.php M X DashboardController.php M PreferensiController.php U preferensi.blade.php U Tr
routes > web.php
2
3 use App\Http\Controllers\ProfileController;
4 use Illuminate\Support\Facades\Route;
5 use App\Http\Controllers\DashboardController;
6 use App\Http\Controllers\TransaksiController;
7 use App\Http\Controllers\PreferensiController;
8
9 Route::get('/', function () {
10 |     return view('welcome');
11 | });
12
13 Route::get(
14 |     '/dashboard',
15 |     [
16 |         DashboardController::class,
17 |         'index'
18 |     ]
19 | )
20 ->middleware([
21 |     'auth',
22 |     'verified'
23 | ])
24 ->name(
25 |     'dashboard'
26 | );
27 Route::post(
28 |     '/reset-kunjungan',
29 |     [
30 |         DashboardController::class,
31 |         'reset'
32 |     ]
33 | )
34 ->name(
35 |     'kunjungan.reset'
36 | );
37 Route::middleware('auth')->group(function() {
38 |     Route::resource(
39 |         'transaksi',
40 |         TransaksiController::class
41 |     );
```

```
web.php M DashboardController.php M X PreferensiController.php U preferensi.blade.php U TransaksiController.php M
app > Http > Controllers > DashboardController.php > DashboardController > reset
7 class DashboardController
9 {
10
11     public function index(
12         Request $request
13     ) {
14         $visit =
15             session(
16                 'visit',
17                 0
18             );
19         $visit++;
20         session([
21             'visit' =>
22                 $visit
23         ]);
24
25         if (
26             !session()->has(
27                 'first_visit'
28             )
29         ) {
30             session([
31                 'first_visit' =>
32                     now()
33             ]);
34         }
35
36         session([
37             'last_visit' =>
38                 now()
39         ]);
40
41         return view(
42             'dashboard',
43             [
44                 'visit' =>
45                     session(
46                         'visit'
47                     )
48             ]
49         );
50     }
51 }
```

Tampilan:





Penjelasan: Fitur ini dibuat sebuah section statistik kunjungan dashboard pada aplikasi BersihIn menggunakan Session Laravel untuk menghitung berapa kali pengguna mengunjungi halaman dashboard. Setiap kali halaman dibuka atau direfresh, sistem akan memeriksa apakah data kunjungan sudah tersimpan pada session. Jika belum ada, sistem akan membuat data awal berupa jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Selanjutnya setiap pengguna kembali mengakses halaman, nilai jumlah kunjungan akan bertambah dan waktu kunjungan terakhir akan diperbarui secara otomatis. Implementasi dilakukan pada DashboardController menggunakan fungsi session() untuk menyimpan dan mengambil data sementara selama sesi pengguna masih aktif. Data yang telah dihitung kemudian dikirim ke halaman dashboard dan ditampilkan dalam bentuk section yang berisi jumlah kunjungan, waktu pertama halaman dibuka, serta waktu terakhir pengguna mengakses halaman. Selain itu, ditambahkan fitur Reset Statistik menggunakan tombol yang terhubung ke method reset() untuk menghapus data session melalui session()->forget() sehingga perhitungan kunjungan dapat dimulai kembali dari awal. Fitur ini dibuat untuk menampilkan aktivitas penggunaan dashboard secara dinamis tanpa menggunakan database dan sesuai dengan konsep penyimpanan sementara pada Session Laravel.